Cloud Architecture and Capacity Plan

Delivery 3 – Cloud Architecture & Scalability

Moreno Barragán Jhonatan David

Gaitán Castañeda Luna Valeria

Ochoa Silva Jonathan Felipe

Ortiz Luna Nicolás Francisco

October 2026

Contents

1 Executive Summary **4**

2 Cloud Architecture Diagram **6**

2.1 Architecture Overview 6

2.2 Complete AWS Architecture Diagram 6

2.3 Delivery 2 to AWS Resource Mapping 6

2.4 Infrastructure Components and Responsibilities 7

2.5 Architecture Assumptions 7

3 Compute Design **7**

3.1 Compute Platform Selection 7

3.2 Service Deployment and Resource Sizing 7

3.3 Capacity Planning and Instance Count Calculations 8

3.4 Auto-Scaling Policies 8

3.5 Compute High Availability 8

3.6 Sizing Assumptions and Limitations 8

4 Networking Design **8**

4.1 VPC and Addressing Plan 8

4.2 Networking Diagram 10

4.3 Ingress and Load Balancing 10

4.4 Routing and Egress 11

4.5 Security Groups and Network ACLs 11

4.6 Multi-AZ Availability 13

4.7 Network Capacity and Bottlenecks 13

5 Security Design **14**

5.1 Authentication and Token Flow 14

5.2 Authorization and IAM Roles 15

5.3 Secrets Management 17

5.4 Network Security Controls 18

5.5 Encryption and Key Management 18

5.6 Audit, Compliance, and Data Protection 19

5.7 Threats and Mitigations 20

6 Data Infrastructure **22**

6.1 Data Architecture Overview 22

6.2 Managed Data Service Selection 22

6.3 Relational Data Stores 23

6.4 NoSQL and High-Volume Data 23

6.5 Caching Layer 23

6.6 Object Storage 23

6.7 Partitioning and Data Growth 24

6.8 Backup and Disaster Recovery 24

6.9 Data Security and Access 24

7 Communication Infrastructure **25**

7.1 Communication Mapping 25

7.2 Queues, Topics, and Event Buses 25
7.3 Amazon SQS Configuration 25
7.4 Amazon SNS and EventBridge Configuration 26
7.5 Consumer Scaling 26
7.6 Reliability and Idempotency 26
7.7 Communication Diagram 27

8 Operations Plan **27**
8.1 Observability Strategy 27
8.2 Logging 28
8.3 Metrics and Dashboards 28
8.4 Alerting 29
8.5 Health Checks 29
8.6 Distributed Tracing 30

9 Growth Scenario Analysis **30**
9.1 Workload Projection 30
9.2 Compute Bottlenecks 31
9.3 Database Bottlenecks 32
9.4 Networking and Communication Bottlenecks 32
9.5 Architectural Evolution Summary 33

10 Cost Analysis **33**
10.1 Estimation Assumptions 33
10.2 Monthly Cost by Category 34
10.3 Cost Calculation Methodology 35
10.4 Cost Comparison and Interpretation 36

1 Executive Summary

This delivery translates the service-oriented, event-driven platform defined in Delivery 2 into a concrete AWS deployment and verifies that the resulting infrastructure can sustain the workload established in Delivery 1. The eight microservices SVC-01 through SVC-08 keep the same responsibilities, communication patterns, and data boundaries; what changes is the specific set of managed services that now support them, together with the networking, security, observability, and capacity decisions that production operation requires.

Workload baseline. Delivery 1 estimates an average load of 45 million requests per hour (12,500 requests per second) and a peak of 120 million requests per hour (33,333 requests per second), with a p95 latency target below 500 ms, a 99.99% availability objective, and an initial data footprint of 20 TB growing by 15 TB per year. The growth scenario studied in this delivery multiplies the average load by 20, reaching 900 million requests per hour (250,000 requests per second). Every infrastructure decision in this document is sized against these two reference points.

Region and availability. The platform runs in a single AWS Region (`us-east-1`, N. Virginia) deployed across three Availability Zones. Three zones leave 66% of the capacity available after the loss of one zone, which is the headroom the 99.99% availability target requires. All stateful services (Amazon RDS for PostgreSQL, Amazon ElastiCache for Redis, Amazon DynamoDB) replicate across zones by configuration or by design, and compute is spread evenly over the same three zones.

Compute layer. Stateless application code for every microservice runs on AWS Fargate under Amazon ECS. Fargate removes the need to manage EC2 capacity, scales per task, and prices against vCPU-hours and GB-hours, which aligns with the per-service sizing calculated from the request rates above. Driver telemetry (SVC-04) runs in a dedicated Fargate service behind a Network Load Balancer so that WebSocket connections remain stable during scaling events.

Data layer. Each service owns its own data store. Accounts and drivers (SVC-01), payments (SVC-06), and pricing and geospatial queries (SVC-05) use Amazon RDS for PostgreSQL Multi-AZ, with PostGIS enabled for SVC-05. Ride requests (SVC-02), trips and tracking (SVC-04), feedback (SVC-07), and notifications (SVC-08) use Amazon DynamoDB, which absorbs the write-heavy telemetry workload through horizontal partitioning. Matching and dispatch (SVC-03) use Amazon ElastiCache for Redis in Multi-AZ cluster mode for microsecond reads. Object payloads (documents, exports, attachments) live in Amazon S3 with Block Public Access enabled, SSE-KMS encryption, lifecycle rules for cost tiering, and cross-region replication for disaster recovery.

Communication layer. Synchronous client traffic enters through Amazon API Gateway and the Application Load Balancer, and is forwarded to the appropriate Fargate target group. Asynchronous events use the Amazon SNS to Amazon SQS fan-out pattern, which decouples producers from consumers and allows each subscriber to process events at its own rate. Payment events require exact ordering and therefore use Amazon SQS FIFO. Third-party webhooks arrive through Amazon EventBridge after signature validation, and rule-based routing delivers them to the owning service. Every primary queue has an attached dead-letter queue, and consumers are idempotent so that the at-least-once delivery semantics of SQS Standard do not translate into duplicated side effects.

Networking and security. A single Amazon VPC with CIDR `10.0.0.0/16` provides public subnets for load balancers and NAT Gateways, private application subnets for Fargate tasks, and private data subnets for databases and caches, all replicated across the three Availability Zones. One NAT Gateway per zone avoids a single point of failure and keeps egress traffic within the local zone. Gateway VPC endpoints for Amazon S3 and Amazon DynamoDB, together with interface endpoints for Secrets Manager, KMS, SQS, SNS, CloudWatch Logs, and ECR, keep AWS service traffic inside the AWS network. End-user authentication uses Amazon Cognito User Pools with short-lived JWT access tokens validated at API Gateway. Each service runs under a least-privilege IAM task role whose policies name resources explicitly. AWS Secrets Manager stores every sensitive credential with automatic rotation for databases, AWS WAF protects the public entry point against the OWASP Top 10 and abusive traffic, and AWS KMS Customer Managed Keys encrypt every data store, with a dedicated stricter-policy key for payment data.

Observability and operations. Amazon CloudWatch Logs collects structured application logs under `/aws/ride-sharing/SVC-XX` with 30-day retention for operational logs and 365 days for security logs. Container Insights supplies infrastructure metrics, CloudWatch Alarms detect threshold breaches on latency, error rate, resource saturation, and queue depth, and AWS X-Ray provides distributed tracing across API Gateway, Fargate services, and asynchronous workflows. Every service exposes `/health/live` and `/health/ready` endpoints that the ALB and the ECS agent use to replace unhealthy tasks automatically. These controls jointly support the 99.99% availability target and the p95 latency target under 500 ms set in Delivery 1.

Growth to 20× load. At 250,000 requests per second the telemetry workload alone reaches 150,000 requests per second under the 60% share estimated in Delivery 2. The architecture absorbs this growth by scaling Fargate tasks horizontally, by distributing DynamoDB write activity through well-chosen partition keys, by adding read replicas and connection pooling in RDS, and by leaning on the queue-based consumer scaling already described for asynchronous workflows. Stateful matching (SVC-03) and write-intensive telemetry (SVC-04) are the two components that may require architectural refinement

rather than linear scaling, and the design accommodates that refinement through geographic partitioning and batched processing.

Cost drivers and assumptions. The main cost drivers are Fargate vCPU-hours and GB-hours, RDS instance-hours with Multi-AZ, DynamoDB read and write capacity, data transfer across Availability Zones, CloudWatch log ingestion, and NAT Gateway processed-data charges. The baseline estimate uses On-Demand pricing, 730 operating hours per month, and no promotional credits. Reserved pricing, Savings Plans, and architectural optimizations (for example, additional Gateway endpoints and tighter log sampling) are explicitly left as levers for the final cost report once resource quantities have been exercised under load testing.

The sections that follow document each of these decisions in detail, present the sizing arithmetic for compute and capacity, describe the networking and security posture, and estimate the resulting monthly cost under the initial and 20× scenarios.

2 Cloud Architecture Diagram

2.1 Architecture Overview

2.2 Complete AWS Architecture Diagram

Figure 1: Complete AWS Cloud Architecture

2.3 Delivery 2 to AWS Resource Mapping

Delivery 2 Component	**AWS Resource**	**Deployment Rationale**
SVC-01 – Account and Driver Management	TODO	TODO
SVC-02 – Ride Request Management	TODO	TODO
SVC-03 – Matching and Dispatch	TODO	TODO

SVC-04 – Trip Management and Tracking	TODO	TODO
SVC-05 – Pricing and Geospatial	TODO	TODO
SVC-06 – Payment and Settlement	TODO	TODO
SVC-07 – Feedback and Safety	TODO	TODO
SVC-08 – Notification and Communication	TODO	TODO

2.4 Infrastructure Components and Responsibilities

2.5 Architecture Assumptions

3 Compute Design

3.1 Compute Platform Selection

3.2 Service Deployment and Resource Sizing

Service	**Compute**	**vCPU**	**Memory**	**State**	**Initial Tasks**
SVC-01	TODO	TODO	TODO	TODO	TODO
SVC-02	TODO	TODO	TODO	TODO	TODO
SVC-03	TODO	TODO	TODO	TODO	TODO
SVC-04	TODO	TODO	TODO	TODO	TODO
SVC-05	TODO	TODO	TODO	TODO	TODO
SVC-06	TODO	TODO	TODO	TODO	TODO
SVC-07	TODO	TODO	TODO	TODO	TODO
SVC-08	TODO	TODO	TODO	TODO	TODO

3.3 Capacity Planning and Instance Count Calculations

The initial workload model defines 45,000,000 requests per hour under average conditions and 120,000,000 requests per hour at peak.

$$R_{\text{average}} = \frac{45{,}000{,}000}{3600} = 12{,}500 \text{ requests/second}$$

$$R_{\text{peak}} = \frac{120{,}000{,}000}{3600} \approx 33{,}333.33 \text{ requests/second}$$

$$N_{\text{required}} = \left\lceil \frac{R_{\text{service, peak}}}{R_{\text{task, sustainable}}} \right\rceil$$

3.4 Auto-Scaling Policies

Service	Scaling Metric	Scale-Out	Scale-In	Cooldown
SVC-01	TODO	TODO	TODO	TODO
SVC-02	TODO	TODO	TODO	TODO
SVC-03	TODO	TODO	TODO	TODO
SVC-04	TODO	TODO	TODO	TODO
SVC-05	TODO	TODO	TODO	TODO
SVC-06	TODO	TODO	TODO	TODO
SVC-07	TODO	TODO	TODO	TODO
SVC-08	TODO	TODO	TODO	TODO

3.5 Compute High Availability

3.6 Sizing Assumptions and Limitations

4 Networking Design

4.1 VPC and Addressing Plan

The platform is deployed in a single Amazon VPC in the `us-east-1` Region (N. Virginia). This region offers the broadest availability of the managed services selected in the

Data Infrastructure and Communication Infrastructure sections, together with competitive pricing and a long history of service maturity. The Region is accessed from Latin America with acceptable round-trip latency for the mobile and web traffic described in Delivery 1, and it provides at least six Availability Zones, which gives the architecture enough physical separation to meet the 99.99% availability target.

The VPC uses the private CIDR block `10.0.0.0/16`, which provides 65,536 IP addresses. This range is intentionally large: it supports the growth of ECS Fargate tasks under the 20× scenario (where each task consumes one elastic network interface and therefore one private IP address), reserves room for additional VPC endpoints, and leaves space for future peering with corporate or analytics accounts without renumbering.

The VPC is deployed across three Availability Zones (`us-east-1a`, `us-east-1b`, `us-east-1c`). Using three zones instead of two keeps 66% of the capacity available when a zone is lost, which is consistent with the resilience assumption in Section 9. Each AZ receives one public subnet for internet-facing load balancers and NAT Gateways, one private application subnet for ECS Fargate tasks, and one private data subnet for RDS, ElastiCache, and interface VPC endpoints.

Subnet	**CIDR Block**	**Purpose**
Public subnet AZ-A	10.0.0.0/22	Internet-facing ALB, NLB, and NAT Gateway for AZ-A.
Public subnet AZ-B	10.0.4.0/22	Internet-facing ALB, NLB, and NAT Gateway for AZ-B.
Public subnet AZ-C	10.0.8.0/22	Internet-facing ALB, NLB, and NAT Gateway for AZ-C.
Private application subnet AZ-A	10.0.16.0/20	ECS Fargate tasks for SVC-01 through SVC-08 in AZ-A.
Private application subnet AZ-B	10.0.32.0/20	ECS Fargate tasks for SVC-01 through SVC-08 in AZ-B.
Private application subnet AZ-C	10.0.48.0/20	ECS Fargate tasks for SVC-01 through SVC-08 in AZ-C.
Private data subnet AZ-A	10.0.64.0/22	RDS, ElastiCache, RDS Proxy, interface VPC endpoints for AZ-A.
Private data subnet AZ-B	10.0.68.0/22	RDS, ElastiCache, RDS Proxy, interface VPC endpoints for AZ-B.
Private data subnet AZ-C	10.0.72.0/22	RDS, ElastiCache, RDS Proxy, interface VPC endpoints for AZ-C.

Table 4: Proposed VPC subnet plan for `10.0.0.0/16` in `us-east-1`.

The three application subnets together provide 12,288 private addresses. At peak demand, the Compute Design section estimates roughly 33,333 requests per second across the eight

services; assuming an average sustainable rate of 100 requests per second per Fargate task, the platform runs on the order of 400 to 600 tasks at peak, well within the available address space. The remaining capacity absorbs the 20× growth scenario without requiring a second VPC.

4.2 Networking Diagram

Figure 2: VPC layout in `us-east-1` with public subnets for load balancers and NAT Gateways, private application subnets for ECS Fargate tasks, private data subnets for managed databases, and VPC endpoints for AWS service access.

4.3 Ingress and Load Balancing

External traffic enters the platform through Amazon API Gateway for REST APIs and through a dedicated Application Load Balancer (ALB) for the mobile and web clients that connect directly to HTTPS endpoints. Both entry points terminate TLS using certificates managed by AWS Certificate Manager, and both reside in the public subnets of all three Availability Zones.

The ALB routes traffic to ECS Fargate target groups that run in the private application subnets. Listener rules match incoming paths such as `/accounts/*`, `/rides/*`, or `/payments/*` and forward them to the appropriate service target group. Each service exposes HTTPS on port 443 to the ALB and HTTP on an internal port (for example, 8080) to its own target group. Idle connection timeout is set to 60 seconds, which is long enough for typical request-response cycles while bounding the number of half-open connections a client can maintain.

SVC-04 (Trip Management and Tracking) requires persistent WebSocket connections for driver telemetry. These connections use a Network Load Balancer (NLB) in the same public subnets. NLB preserves client source addresses and provides a stable TCP endpoint for the WebSocket upgrade, with lower connection-handling overhead than an ALB for long-lived streams. The NLB forwards traffic to a dedicated ECS Fargate service for telemetry, which is horizontally scaled according to the open-connection count in addition to the queue-depth metric defined in the Communication Infrastructure section.

Internal service-to-service calls that do not need to leave the VPC use internal ALBs attached to the private application subnets. These internal load balancers never receive internet traffic, and their security groups accept connections only from the security group of the calling service.

4.4 Routing and Egress

The VPC contains one internet gateway attached to the public route table. The public route table forwards `0.0.0.0/0` traffic to the internet gateway, while keeping the `10.0.0.0/16` route for local VPC reachability.

Outbound internet traffic from private subnets uses NAT Gateways. One NAT Gateway is deployed in each Availability Zone, and the private application subnet in that AZ has its own route table that sends `0.0.0.0/0` to the local NAT Gateway. Deploying one NAT Gateway per zone avoids a single point of failure during an AZ outage and prevents cross-AZ data transfer charges for routine egress.

Access to AWS services uses VPC endpoints whenever possible to keep traffic inside the AWS network, reduce NAT Gateway cost, and simplify security group policies. Two Gateway endpoints are configured for Amazon S3 and Amazon DynamoDB; these endpoints are free of hourly charges and do not count against NAT bandwidth. Interface (PrivateLink) endpoints are configured for AWS Secrets Manager, AWS KMS, Amazon CloudWatch Logs, Amazon SQS, Amazon SNS, and Amazon ECR. The private data subnets are intentionally kept without a route to NAT, which guarantees that database and cache workloads cannot reach the public internet by accident.

4.5 Security Groups and Network ACLs

Security groups form the primary network boundary within the VPC. All groups default to deny, and inbound rules reference other security groups by ID rather than CIDR blocks whenever both endpoints live inside the VPC. This keeps rules stable as tasks scale and move between AZs. Network ACLs are left in their default allow-all configuration to avoid stateless rule duplication, except for the data subnet NACL, which explicitly denies traffic to `0.0.0.0/0` as a second layer of defense.

Source Tier	Destination Tier	Port/Protocol	Rule and Justification
Internet (0.0.0.0/0)	Public ALB (SG-ALB)	443/TCP	Public HTTPS entry point. Port 80 is accepted only to issue an HTTP 301 redirect to HTTPS, enforced at listener level.

Internet (0.0.0.0/0)	Public NLB (SG-NLB)	443/TCP	Driver telemetry WebSocket over TLS. TCP is required to preserve long-lived streams for SVC-04.
SG-ALB	ECS task SG (SG-APP)	8080/TCP	ALB forwards HTTP traffic to Fargate tasks on the service port. Inbound is permitted only from the ALB security group, never from the internet or cross-service callers.
SG-APP	RDS SG (SG-DB)	5432/TCP	Only services with a documented owner of a PostgreSQL schema may open connections to the corresponding RDS instance. Database connections pass through RDS Proxy.
SG-APP	ElastiCache SG (SG-CACHE)	6379/TCP	Only SVC-03 (Matching and Dispatch) and SVC-05 (Pricing and Geospatial) may reach the Redis cluster.
SG-APP	VPC endpoint SG (SG-ENDPOINT)	443/TCP	Access to Secrets Manager, KMS, SQS, SNS, CloudWatch Logs, and ECR through interface endpoints.
SG-APP	0.0.0.0/0 via NAT	443/TCP	Egress to external APIs (payment provider, mapping provider, push notification services). No other outbound ports are allowed.

Administrative access to Fargate tasks is provided by AWS Systems Manager Session

Manager and ECS Exec. This removes the need for a bastion host, SSH keys, and inbound port 22, while still allowing operators to open an interactive shell into a running container for troubleshooting.

4.6 Multi-AZ Availability

Every tier of the architecture spans at least two Availability Zones, and the production configuration uses three. The Application Load Balancer and Network Load Balancer are deployed with cross-zone load balancing enabled, so the loss of a zone does not bias traffic toward the remaining zones. ECS Fargate services use the `spread` placement strategy across AZs, ensuring that no single AZ runs a disproportionate share of a service's tasks.

Managed data services replicate automatically across zones as described in the Data Infrastructure section: RDS runs Multi-AZ with synchronous replication to a standby in a second AZ, ElastiCache Redis uses Multi-AZ with automatic failover, and DynamoDB replicates across all AZs in the Region by design. NAT Gateways are deployed one per AZ, so an application task never routes egress through another zone's NAT.

In the failure scenario where `us-east-1b` becomes unreachable, the ALB stops routing traffic to its targets in that zone, Fargate replaces failed tasks in `us-east-1a` and `us-east-1c`, the RDS standby in a surviving zone is promoted to primary, and the ElastiCache replica in a surviving zone is promoted. The system continues to serve traffic with roughly 66% of the original compute footprint, which is why the Compute Design section sizes the service baseline with at least 50% headroom beyond the measured average load.

4.7 Network Capacity and Bottlenecks

Three potential bottlenecks guide capacity planning at the network layer. The first is NAT Gateway throughput. A single NAT Gateway supports up to 100 Gbps of bandwidth and 55,000 simultaneous connections per unique destination [1]. Routing all egress through NAT would place the external payment and mapping APIs close to the per-destination connection limit under the 20× scenario, which is why VPC endpoints absorb AWS-service traffic and leave NAT reserved for third-party APIs.

The second is cross-zone data transfer. Communication between Availability Zones is billed per gigabyte in both directions. The design mitigates this cost in three ways: ECS services prefer same-zone targets when they communicate through internal ALBs with zonal DNS enabled; DynamoDB and S3 traffic uses Gateway endpoints that stay within the Region without crossing zones; and Multi-AZ RDS replication traffic is unavoidable

[1] https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html

but bounded by the write workload rather than by request volume.

The third is load balancer capacity. The Application Load Balancer scales automatically, but under sudden spikes larger than a doubling in a few minutes AWS recommends requesting a pre-warming through a support case. This is an operational concern for events such as promotional campaigns and must be coordinated with the growth scenario in Section 9. The Network Load Balancer handles millions of concurrent connections per Availability Zone without pre-warming, which is why driver telemetry sits behind an NLB rather than an ALB.

VPC Flow Logs are enabled at the subnet level with aggregated five-minute windows. The logs are delivered to the operations log group described in Section 8 and feed anomaly-detection queries for unexpected egress or rejected flows.[2]

5 Security Design

5.1 Authentication and Token Flow

End-user authentication is delegated to Amazon Cognito. The platform configures two separate User Pools: one for riders and one for drivers. Keeping the pools separate allows each population to have its own sign-up and sign-in flows, its own password and multi-factor policies, and its own attribute schema. Drivers additionally use the identity-verification attributes that SVC-01 (Account and Driver Management) populates during onboarding, while riders use a lighter-weight profile.

Both pools issue OAuth 2.0 and OpenID Connect tokens. A successful sign-in returns three tokens: an ID token (identity claims), an access token (used to authorize API calls), and a refresh token. Access tokens have a short lifetime of 15 minutes to limit the damage of a leaked token; refresh tokens have a 30-day sliding window so that typical mobile users do not need to re-enter credentials every two weeks. Mobile clients store refresh tokens in the platform's secure keystore (iOS Keychain or Android Keystore), never in local files or shared preferences.

API Gateway and the Application Load Balancer validate tokens at the edge. API Gateway uses a Cognito authorizer that verifies the token signature against the pool's JSON Web Key Set, checks the expiration, and extracts the authenticated subject (`sub`) and the `cognito:groups` claim. The authorizer rejects requests with missing, expired, or malformed tokens before they reach ECS Fargate tasks, which protects downstream services from unauthenticated traffic. Every service still validates the token locally for defense in depth, using the JWKS cached inside the task.

[2] https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html

Service-to-service calls that stay inside the VPC do not use user tokens. Instead, each ECS task runs under an IAM task role, and the calling service signs requests with Signature Version 4 when the target is an AWS API. For direct HTTP calls between internal services, the calling task adds a short-lived SigV4-signed header that the receiving service validates against the caller's expected task role. Third-party providers (payment processor, mapping provider, push notification providers) authenticate outbound requests with API keys that are stored in AWS Secrets Manager and loaded at task startup. Incoming webhooks from these providers reach Amazon EventBridge through API Gateway with signature validation at the gateway layer before an EventBridge rule routes the event to its handler.

5.2 Authorization and IAM Roles

Authorization follows the least-privilege principle. Every service runs under a dedicated ECS task role, and every task role grants only the specific actions on the specific resources that service requires. The ECS task execution role (used by the agent to pull images and write logs) is separate from the task role (used by the running application code). Separating these two roles ensures that application code never inherits the broad permissions required by the execution plane.

Principal/Service	**Role or Policy**	**Minimum Required Permissions**
SVC-01 Accounts/Drivers	`role-svc01-task`	`rds-db:connect` to the accounts database, `s3:PutObject` and `s3:GetObject` on the driver-documents prefix, `cognito-idp:AdminUpdateUserAttributes` limited to the Drivers User Pool, `secretsmanager:GetSecretValue` on the SVC-01 secret.

SVC-02 Ride Requests	`role-svc02-task`	`dynamodb:PutItem`, `Query`, `UpdateItem` on the ride-requests table, `sns:Publish` on the ride-requested topic, `kms:Decrypt` on the request-data CMK.
SVC-03 Matching and Dispatch	`role-svc03-task`	`elasticache:Connect` to the matching cluster, `sqs:ReceiveMessage` and `DeleteMessage` on the matching queue, `dynamodb:Query` on the drivers-availability table.
SVC-04 Trips and Tracking	`role-svc04-task`	`dynamodb:PutItem`, `Query`, `UpdateItem` on the trips and telemetry tables, `sns:Publish` on the trip-completed topic, `s3:PutObject` on the telemetry-archive bucket.
SVC-05 Pricing and Geospatial	`role-svc05-task`	`rds-db:connect` to the PostGIS instance (read-only replica preferred), `elasticache:Connect` for fare cache.
SVC-06 Payments and Settlement	`role-svc06-task`	`rds-db:connect` to the payments database, `sqs:ReceiveMessage` and `DeleteMessage` on the payments FIFO queue, `events:PutEvents` on the payments event bus, `kms:Decrypt` and `kms:Encrypt` on the payments CMK.

SVC-07 Feedback and Safety	`role-svc07-task`	`dynamodb:PutItem` and `Query` on the feedback table, `s3:PutObject` on the safety-attachments bucket, `sns:Publish` on the safety-incident topic.
SVC-08 Notifications	`role-svc08-task`	`sqs:ReceiveMessage` on the notification queues, `sns:Publish` on the SMS and email topics, `secretsmanager:GetSecretValue` on the push-provider credentials.

Every resource in the policies above is identified by its full Amazon Resource Name, which prevents a service from accidentally operating against resources that belong to another service. IAM Access Analyzer runs on a continuous basis to flag any policy that grants broader access than the actions in use require. No long-lived access keys are issued to tasks, pipelines, or human operators: tasks use task roles, pipelines use GitHub OIDC with role assumption, and human operators use AWS IAM Identity Center with short-lived credentials.

5.3 Secrets Management

AWS Secrets Manager stores every piece of sensitive configuration that services need at runtime. This includes RDS and ElastiCache credentials, payment-provider API keys, push-notification provider keys, and the signing keys used by SVC-06 to verify webhook signatures. Each secret is scoped to a single service, and the resource-based policy on the secret allows only that service's task role to call `GetSecretValue`.

Credentials for RDS are generated by Secrets Manager and rotated automatically every 30 days using the managed rotation lambda. During rotation the previous password remains valid for a short overlap window so that in-flight connections do not fail. Third-party API keys are rotated manually on a quarterly schedule or immediately after any suspected exposure, and the operational runbook includes the exact sequence of steps for each provider.

Secrets are never embedded in container images, environment variables defined at build time, or configuration files committed to Git. The ECS task definition references each se-

cret by ARN, and the Fargate agent injects the current value into the container at startup through the task role, not through a leaked environment. Every call to `GetSecretValue` is recorded in AWS CloudTrail, and an unexpected caller or an unexpected secret triggers an alarm in the operations dashboard.

5.4 Network Security Controls

Network security combines the layer-3/4 controls defined in the Networking Design section with application-layer controls at the edge. AWS WAF is attached to the public Application Load Balancer and to the API Gateway stage. WAF enforces the AWS managed rule sets for the OWASP Top 10 (Core Rule Set, Known Bad Inputs, SQL Injection, and Linux-specific rules), together with a rate-based rule that blocks any source IP exceeding 5,000 requests in a five-minute window. Geographic restrictions can be enabled on short notice if an abuse campaign is traced to a specific region.

AWS Shield Standard is enabled by default and absorbs common network-layer and transport-layer DDoS attacks against the ALB, NLB, and CloudFront distributions used for static assets. Shield Advanced is not enabled in the initial phase because its cost is significant; the design leaves the door open for upgrading if observed attacks justify it.

All application and data tiers live in private subnets with no inbound route from the internet. Interactive access to containers uses AWS Systems Manager Session Manager and ECS Exec rather than SSH, so no bastion host is required and no inbound port 22 is ever opened. Human administrators receive temporary credentials from AWS IAM Identity Center and assume a least-privilege role to perform operational tasks; the role's maximum session duration is one hour.

5.5 Encryption and Key Management

Data at rest is encrypted with keys managed by AWS Key Management Service. Each service owns at least one Customer Managed Key (CMK), and the key policy limits `kms:Decrypt` and `kms:Encrypt` to the owning service's task role and to the specific AWS service principals (for example, `rds.amazonaws.com`, `dynamodb.amazonaws.com`, `s3.amazonaws.com`) that must call KMS on the service's behalf. Payment data (SVC-06) uses a dedicated CMK with a stricter policy and separate rotation schedule because its disclosure would have the largest impact. Automatic annual rotation is enabled on every CMK, and AWS maintains the full backing-key history so that previously encrypted data remains readable after rotation.

RDS, DynamoDB, ElastiCache, S3, and EBS volumes attached to any instance store

use KMS-based encryption at rest, as documented in the Data Infrastructure section. CloudWatch Logs and Secrets Manager are also encrypted with service-specific CMKs so that privileged readers of those services cannot bypass per-service boundaries.

Data in transit is encrypted with TLS 1.2 or higher on every external endpoint and on every internal link. The ALB and NLB terminate TLS using certificates issued by AWS Certificate Manager; ACM renews these certificates automatically and attaches them to the load balancers without downtime. Service-to-service HTTP calls inside the VPC also run over TLS, with certificates issued by AWS Private CA for internal names. VPC endpoint connections use TLS by default, so traffic between application tasks and Secrets Manager, KMS, SQS, or SNS never leaves AWS and is never in plaintext.

Field-level encryption adds a third layer of protection for the most sensitive data. SVC-06 encrypts payment tokens and the last four digits of card numbers at the application layer with the AWS Encryption SDK before storing them in RDS, using a data key derived from the payments CMK. This means that even a privileged reader of the database still cannot reconstruct a usable payment identifier without also obtaining permission to call KMS on the payments CMK.

5.6 Audit, Compliance, and Data Protection

A multi-region AWS CloudTrail trail records every API call made in the account. The trail delivers logs to a dedicated S3 bucket with SSE-KMS encryption, Block Public Access enabled, versioning, and S3 Object Lock in governance mode to prevent tampering. Log file integrity validation is enabled so that any later modification to a delivered log file can be detected.

AWS Config tracks configuration changes across all supported resources and evaluates the inventory against a conformance pack that includes rules such as `s3-bucket-public-read-prohibited`, `rds-storage-encrypted`, `incoming-ssh-disabled`, `iam-password-policy`, and `wafv2-logging-enabled`. Amazon GuardDuty is enabled in every active Region and feeds its findings to the operations alerting pipeline described in the Operations Plan section.

Personal data is classified at design time following the categories identified in Delivery 1: account identifiers, contact information, government-issued documents, geolocation traces, and payment information. S3 buckets that contain documents or exports apply lifecycle policies that transition objects to lower-cost storage classes after 90 days and expire them after the retention period required by the applicable regulation. DynamoDB telemetry tables use Time-to-Live attributes to automatically expire rows, and RDS backups are deleted after 35 days unless they are placed in a long-term archive. Access reviews

are performed quarterly, and the review covers IAM principals, Secrets Manager resource policies, S3 bucket policies, and KMS key policies.

The design does not claim certification against any specific compliance framework. It does, however, align with the control families that would be examined by an audit: identity and access management, cryptographic protection, logging and monitoring, configuration management, and incident response.

5.7 Threats and Mitigations

The table below lists the main threats identified during design and the controls applied to mitigate them. Every row also names the verification method that confirms the control is active in production.

Threat	Mitigation	Verification Method
Credential exposure	Secrets Manager with per-service resource policies, automatic rotation for RDS every 30 days, no secrets in images or environment variables defined at build time, CloudTrail alarms on unexpected `GetSecretValue` callers.	Quarterly review of secret access logs, periodic simulated rotation, automated scans for committed secrets in the code repository.
Unauthorized API access	Cognito User Pools with short-lived access tokens, API Gateway Cognito authorizer, in-service JWT validation, IAM task roles for service-to-service calls, AWS WAF rate-based rules on the public edge.	Authorization tests in CI for every endpoint, periodic review of WAF-blocked requests, GuardDuty findings on anomalous API activity.

Threat	Mitigation	Verification Method
Public data-store exposure	Private subnets without a route to the internet, Block Public Access enabled on every S3 bucket, bucket policies that deny unencrypted access, security groups that reference other security groups by ID, VPC endpoints for AWS service traffic.	AWS Config rules `s3-bucket-public-read-prohibited` and `s3-bucket-public-write-prohibited`, IAM Access Analyzer public-access findings, quarterly review of bucket and database policies.
Traffic abuse and DDoS	AWS Shield Standard across ALB, NLB, and CloudFront; AWS WAF rate-based rules; API Gateway throttling with per-client quotas; auto-scaling queue consumers to drain bursts without dropping messages.	Load tests that simulate abusive clients, dashboards that report WAF blocks and throttled requests, Shield metrics in CloudWatch.
Injection attacks (SQL, OS, NoSQL)	Managed WAF rule sets for SQL injection and known bad inputs, parameterized database queries in every service, input validation at API Gateway models and inside the service, least-privilege database roles.	Static analysis in CI, periodic dynamic application security testing, WAF logs reviewed weekly during the first phase.
Insider threat and privileged misuse	AWS IAM Identity Center with short-lived credentials, separation of duties between the payments role and every other role, mandatory multi-factor authentication for every human principal, CloudTrail with Object Lock for tamper-evident logs.	Quarterly access reviews, monthly report of privileged actions, alarms on sensitive IAM changes (policy attach, role trust edits).

The above controls are applied consistently to every microservice. The threat model will be reviewed after the first penetration test and after each incident, and mitigation and

verification methods will be updated accordingly.

6 Data Infrastructure

6.1 Data Architecture Overview

The cloud data architecture maps the polyglot persistence strategy defined in Delivery 2 directly to AWS managed services. To strictly enforce data ownership and avoid the shared database anti-pattern, every microservice operates its own isolated data store. Direct cross-service database access is prohibited at the network level using VPC Security Groups.

6.2 Managed Data Service Selection

Service/Domain	**AWS Service**	**Configuration**	**Rationale**
SVC-01: Accounts/Drivers	Amazon RDS (PostgreSQL)	Multi-AZ, db.r6g.large	Requires strict ACID compliance for user credentials and onboarding state.
SVC-02: Ride Requests	Amazon DynamoDB	Provisioned + Auto Scaling	High-throughput, flexible schema for varying request payloads.
SVC-03: Matching/Dispatch	Amazon ElastiCache (Redis)	Cluster Mode Enabled	Microsecond reads/writes required for geospatial evaluation.
SVC-04: Trips/Tracking	Amazon DynamoDB	Provisioned capacity	Write-heavy telemetry ingestion (Time-series data).
SVC-05: Pricing/Geospatial	Amazon RDS (PostGIS)	Multi-AZ, Read Replicas	Requires spatial extensions (PostGIS) for complex geofence queries.
SVC-06: Payments/Settlement	Amazon RDS (PostgreSQL)	Multi-AZ, db.r6g.xlarge	Strict financial consistency, auditability, and ACID transactions.

SVC-07: Feedback/Safety	Amazon DynamoDB	On-Demand capacity	Unstructured text, variable metadata, and unpredictable load spikes.
SVC-08: Notifications	Amazon DynamoDB	On-Demand capacity	Key-Value access pattern for delivery logs and templates.

6.3 Relational Data Stores

We utilize Amazon RDS for PostgreSQL for domains requiring strong consistency (SVC-01, SVC-05, SVC-06). All RDS instances are deployed in a Multi-AZ configuration to ensure high availability and automatic failover. SVC-05 (Pricing) utilizes Amazon RDS Read Replicas to offload read-heavy fare estimation traffic. Connection pooling is managed via Amazon RDS Proxy to prevent connection exhaustion during peak demand surges.

6.4 NoSQL and High-Volume Data

Amazon DynamoDB serves high-volume, highly scalable domains (SVC-02, SVC-04). For the telemetry ingestion in SVC-04, the Partition Key is `trip_id` and the Sort Key is `timestamp`. To prevent hot partitions, UUIDs are used to ensure uniform data distribution across underlying storage nodes. We apply DynamoDB Time-to-Live (TTL) to automatically archive stale location data to S3, keeping the active tables lean.

6.5 Caching Layer

Amazon ElastiCache for Redis powers SVC-03 (Matching). The cluster operates in Multi-AZ with automatic failover, utilizing an `r6g` node type. A `volatile-lru` eviction policy is configured since the cache stores transient driver availability sessions. ElastiCache acts as a low-latency operational view, not as the authoritative source of truth for durable entity states.

6.6 Object Storage

Amazon S3 is utilized for storing unstructured payloads such as driver identity documents (SVC-01) and support ticket attachments (SVC-07). Buckets are private by default (Block Public Access enabled) and encrypted at rest using SSE-KMS. We enforce lifecycle rules

to transition documents older than 1 year to S3 Glacier Instant Retrieval for cost optimization.

6.7 Partitioning and Data Growth

DynamoDB natively handles partitioning based on the partition key. Because the system's baseline generates continuous high-frequency telemetry (approx. 45M requests/hour), the choice of `trip_id` guarantees that I/O operations are evenly spread. As data volume grows towards the projected 15 TB/year, horizontal scaling through DynamoDB's auto-scaling mechanisms absorbs the load seamlessly without manual sharding interventions.

6.8 Backup and Disaster Recovery

Data Domain	**Backup Strategy**	**Recovery Target**
Accounts and drivers	Automated daily snapshots + PITR (RDS)	RPO: 5 mins, RTO: 15 mins
Trips and tracking	DynamoDB PITR enabled	RPO: 1 sec, RTO: 15 mins
Payments and settlement	RDS Multi-AZ + Cross-region snapshots	RPO: 5 mins, RTO: 15 mins
Feedback and safety	DynamoDB PITR enabled	RPO: 1 sec, RTO: 15 mins
Objects and exports	S3 Versioning + Cross-Region Replication	RPO: $<$ 1 min, RTO: 5 mins

Table 9: Backup and recovery plan for managed data stores.

Both RDS and DynamoDB leverage Point-In-Time Recovery (PITR), allowing the system to restore the database to any exact second within the last 35 days, protecting against accidental deletions or logical corruption.

6.9 Data Security and Access

All data stores enforce encryption at rest using AWS Key Management Service (KMS) Customer Managed Keys (CMKs) and encryption in transit via TLS 1.2. IAM Roles restrict access, ensuring an ECS task running SVC-04 can only assume a role granting `dynamodb:PutItem` to its specific telemetry table.

7 Communication Infrastructure

7.1 Communication Mapping

The hybrid communication strategy from Delivery 2 maps to AWS managed networking and messaging services. Synchronous API calls (e.g., Rider app requesting a fare estimate) route through the Application Load Balancer (ALB) directly to the target ECS service. Asynchronous events (e.g., Trip Completed) utilize the SNS-to-SQS fan-out pattern to decouple producers from consumers, ensuring background workflows do not impact critical path latency.

7.2 Queues, Topics, and Event Buses

Event/Message	AWS Service	Producer	Consumer(s)	Delivery/Failure
Ride requested	SNS → SQS	SVC-02	SVC-03, SVC-05	Standard, DLQ after 3 retries
Driver telemetry	SQS (Standard)	SVC-04	SVC-03	Standard, high throughput
Trip completed	SNS → SQS	SVC-04	SVC-06, SVC-08	FIFO/Standard, Idempotent
Payment status	EventBridge	SVC-06	SVC-08 (Comms)	Standard, rule-based filtering
Safety incident	SNS → SQS	SVC-07	SVC-01	Standard, DLQ alert

7.3 Amazon SQS Configuration

Amazon SQS Standard queues are utilized for the majority of the event streams (like telemetry) to support virtually unlimited throughput. SQS FIFO (First-In-First-Out) queues are strictly utilized for SVC-06 (Payments) to guarantee exact ordering of financial transactions and prevent double-billing. The Visibility Timeout is set to 3 times the average processing time of the consumer task (e.g., 30 seconds). A Dead-Letter Queue (DLQ) is attached to every primary queue to capture messages that fail processing after 3 maximum receive attempts.

7.4 Amazon SNS and EventBridge Configuration

We implement the *Fan-Out pattern* using Amazon SNS. For example, when SVC-04 publishes a `TripCompleted` event to an SNS Topic, multiple SQS queues (one for Payments, one for Notifications) are subscribed to it. This allows multiple independent microservices to process the exact same event at their own scale and pace. EventBridge is selectively used for routing third-party webhooks (like Payment Provider callbacks) to internal services using pattern-matching rules.

7.5 Consumer Scaling

Worker services in Amazon ECS do not scale based on CPU. Instead, they utilize Target Tracking Scaling Policies based on the custom CloudWatch metric `ApproximateNumberOfMessagesVisi` for their respective SQS queue. If the queue depth exceeds an acceptable threshold (e.g., 100 messages per task), Application Auto Scaling seamlessly provisions additional ECS tasks to clear the backlog, ensuring processing remains highly responsive during traffic surges.

7.6 Reliability and Idempotency

Amazon SQS Standard guarantees at-least-once delivery, meaning consumers might occasionally receive duplicate messages. To maintain data integrity, every consumer implements idempotent processing. For example, SVC-06 (Payments) uses the `trip_id` combined with the event state as a unique constraint in the PostgreSQL database. If a duplicate message arrives, the database rejects the duplicate transaction constraint, and the worker safely discards the message without duplicating the charge.

7.7 Communication Diagram

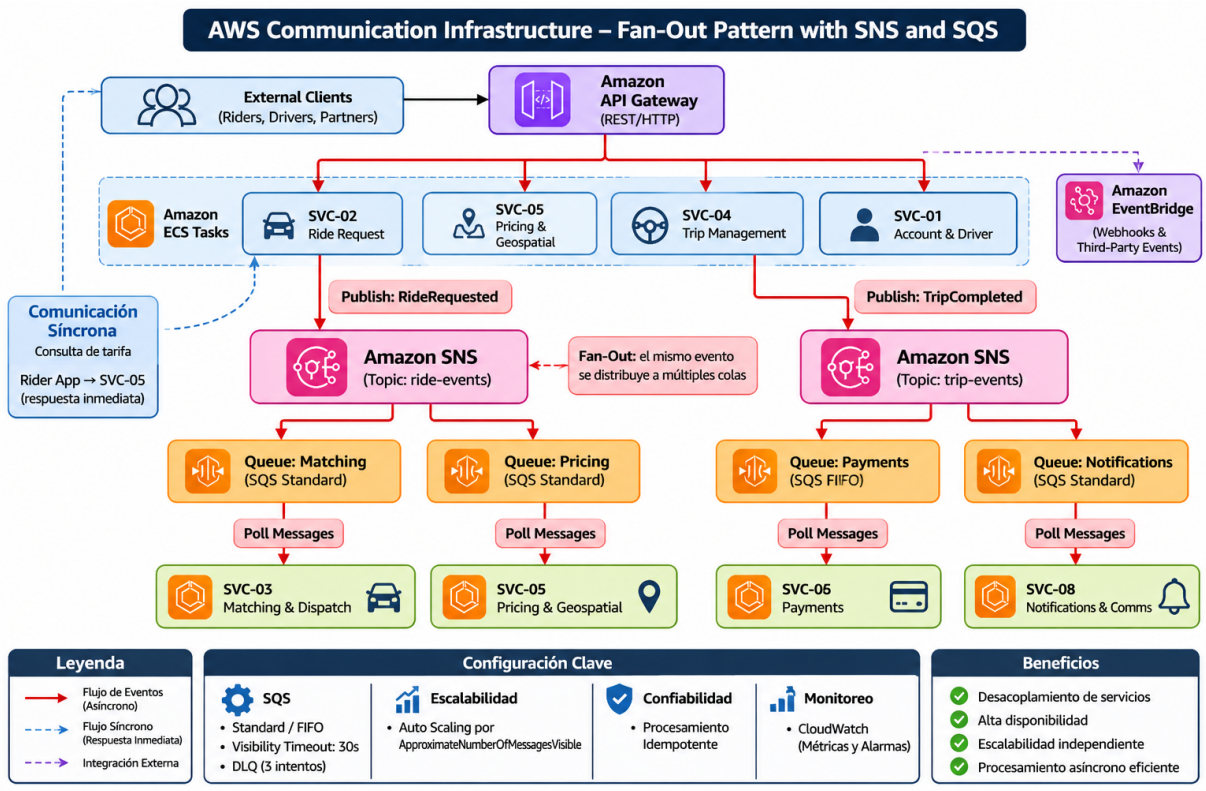


Figure 3: AWS Communication Infrastructure: Synchronous and Asynchronous Communication with SNS-to-SQS Fan-Out Pattern.

8 Operations Plan

8.1 Observability Strategy

The platform adopts a centralized observability strategy based on Amazon CloudWatch and AWS X-Ray. The objective is to monitor the eight microservices independently while maintaining end-to-end visibility across synchronous requests and asynchronous event-driven workflows.

Observability data is correlated through a common request identifier and distributed trace context. This allows operators to identify failures, latency increases, resource saturation, and communication problems without violating the independent deployment and data ownership principles established in Delivery 2.

8.2 Logging

Each microservice will have a dedicated Amazon CloudWatch Logs log group, following the naming convention `/aws/ride-sharing/SVC-XX`. Application logs will use structured JSON records containing, at minimum, a timestamp, log level, service identifier, environment, trace identifier, request identifier, operation, execution time, and result status.

Operational logs will be retained for 30 days. Security and audit logs will use a separate log group with a proposed retention period of 365 days. These periods are initial design assumptions and may be adjusted according to regulatory requirements and the final cost evaluation. Sensitive information, including credentials, access tokens, personal documents, and raw payment data, must not be written to application logs.

CloudWatch Logs supports configurable retention periods at the log-group level; therefore, the proposed policy avoids indefinite storage of operational records and makes log storage costs more predictable.[3]

8.3 Metrics and Dashboards

Amazon CloudWatch dashboards will provide a consolidated view of platform health and service-level performance. Container Insights will collect infrastructure metrics for ECS or EKS workloads, including CPU, memory, network utilization, and container-level diagnostics.[4]

The monitoring strategy will prioritize the following indicators:

- **Performance:** request throughput, average latency, and p95 latency, with a target below 500 ms for latency-sensitive operations.
- **Reliability:** HTTP 5xx error rate, failed requests, service availability, and unhealthy task count.
- **Compute:** CPU and memory utilization, container restarts, and running task count.
- **Data:** database connections, read/write latency, throttled requests, storage utilization, and replication lag where applicable.
- **Messaging:** queue depth, message age, processing failures, and dead-letter queue messages.

[3] https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/Working-with-log-groups-and-streams.html

[4] https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/ContainerInsights.html

Dashboards will be organized by service and by platform-wide indicators. Particular attention will be given to SVC-03 and SVC-04 because matching and telemetry represent the most time-sensitive and traffic-intensive workloads.

8.4 Alerting

CloudWatch Alarms will detect abnormal conditions and notify the operations team through Amazon SNS. Initial alert thresholds are proposed as follows and must be validated through load testing:

- p95 latency exceeding 500 ms for five consecutive minutes on latency-sensitive endpoints.
- HTTP 5xx error rate exceeding 1% over a five-minute period.
- CPU utilization above 70% or memory utilization above 80% for ten minutes.
- Database connection utilization above 80% of the configured limit.
- Persistent queue growth, excessive message age, or any message reaching a dead-letter queue.
- Loss of healthy application targets in an Availability Zone.

Critical alerts will trigger immediate investigation, while warning-level alerts will be used to identify capacity trends before they affect the platform's availability or response-time objectives.

8.5 Health Checks

Application health will be verified at both the load-balancing and container levels. Each service will expose separate endpoints for liveness and readiness checks:

- `/health/live`: confirms that the application process is running and has not entered an unrecoverable state.
- `/health/ready`: confirms that the service can receive traffic and that its essential dependencies are available.

Application Load Balancers will periodically evaluate the health of registered targets and route requests only to healthy targets. Container health checks will additionally allow the compute orchestrator to identify and replace unhealthy tasks.[5]

Health checks must remain lightweight and must not execute expensive database queries or business operations.

8.6 Distributed Tracing

AWS X-Ray will provide distributed tracing across the API Gateway, microservices, and relevant downstream dependencies. Services will propagate trace context through synchronous calls and asynchronous event messages, allowing a single business operation to be followed across service boundaries.

The service map will be used to identify latency bottlenecks, failed dependencies, and communication paths with elevated error rates. Sampling will be enabled to control telemetry volume: a representative fraction of normal requests will be traced, while higher sampling rates may be temporarily applied to critical operations during incident investigation. Health checks and frequent background polling will use low sampling rates to avoid unnecessary tracing overhead.[6]

This strategy supports the platform's 99.99% availability objective and the p95 response-time target defined in Delivery 1.

9 Growth Scenario Analysis

9.1 Workload Projection

The growth scenario evaluates how the proposed AWS architecture must evolve when the platform's average workload increases by a factor of 20. Based on the initial workload model, the platform processes approximately 45 million requests per hour under normal conditions.

The resulting workload is:

$$R_{\text{growth}} = 20 \times 45{,}000{,}000 = 900{,}000{,}000 \text{ requests/hour}$$

Converting this value into requests per second:

[5] https://docs.aws.amazon.com/elasticloadbalancing/latest/application/target-group-health-checks.html

[6] https://docs.aws.amazon.com/xray/latest/devguide/xray-concepts.html

$$R_{\text{growth}} = \frac{900{,}000{,}000}{3600} = 250{,}000 \text{ requests/second}$$

Therefore, the architecture must evolve from approximately 12,500 requests per second to 250,000 requests per second under average operating conditions.

According to Delivery 2, approximately 60% of the initial workload is associated with driver telemetry handled by SVC-04. Assuming that this proportion remains stable, the telemetry workload would increase from approximately 7,500 to 150,000 requests per second. The remaining services would collectively handle approximately 100,000 requests per second.

9.2 Compute Bottlenecks

Stateless services can accommodate increased demand through horizontal scaling by increasing the number of ECS tasks or EKS pods. Auto Scaling policies should react to CPU utilization, memory consumption, request count, and queue depth, depending on each service's workload profile.

However, simply increasing the number of instances may not be sufficient. SVC-03 maintains geographically partitioned operational state, so an uneven distribution of requests could overload specific geographic partitions. SVC-04 also maintains a large number of concurrent real-time connections and processes the majority of telemetry traffic.

To mitigate these limitations, the platform should:

- Increase compute capacity independently for each microservice according to measured throughput.
- Distribute matching and driver availability state across geographic partitions, allowing heavily loaded areas to be scaled separately.
- Use connection-aware scaling and efficient WebSocket handling for SVC-04.
- Apply batching and asynchronous processing where immediate responses are not required.
- Distribute workloads across multiple Availability Zones and preserve sufficient capacity to tolerate the loss of one zone.

Instance counts and task sizes must be recalculated using load-test results rather than assuming that a 20-fold increase in requests requires exactly 20 times the compute resources.

9.3 Database Bottlenecks

The most significant data-layer challenge is the write-intensive workload generated by driver telemetry. A single database instance or a poorly distributed partition key may become a bottleneck as the number of concurrent rides increases.

SVC-04 should use a horizontally scalable, write-optimized storage configuration, partitioning telemetry by geographic area and time interval. High-frequency location updates should be batched when possible, while historical telemetry can be archived in object storage according to retention requirements.

SVC-03 should distribute active driver sessions and matching state across geographic partitions. Partition rebalancing and replication mechanisms will be necessary to prevent overloaded regions from affecting the entire matching system.

For relational workloads, particularly SVC-06 Payment and Settlement, consistency and transaction integrity must remain priorities. Read replicas can reduce read pressure, but they do not eliminate the write capacity limit of the primary database. Database connection pooling, query optimization, appropriate indexing, and controlled write workloads should therefore be applied. If the write workload exceeds the capacity of a single relational primary, the team must evaluate domain-level partitioning or a distributed database design without compromising payment idempotency and auditability.

SVC-01 and SVC-05 should also be evaluated for read scaling, caching, and database capacity as their workloads increase. Frequently accessed pricing and geospatial information can be served through the cache already defined in Delivery 2.

These measures follow AWS guidance on distributing DynamoDB partition-key activity to avoid hot partitions and using RDS read replicas to scale read-heavy workloads.[7][8]

9.4 Networking and Communication Bottlenecks

At 250,000 requests per second, network throughput, connection limits, and cross-zone traffic become important architectural constraints. API Gateway and load balancers must be evaluated against their service quotas and expected concurrent connection counts.

NAT Gateways may also become a cost and throughput concern when large volumes of application traffic are routed through them. The architecture should use VPC endpoints for supported AWS services, distribute outbound traffic across Availability Zones, and avoid unnecessary cross-zone data transfers.

[7] https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-partition-key-design.html

[8] https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_ReadRepl.html

The Event Bus and message queues must support the increased event volume without creating persistent backlogs. Consumers should scale according to queue depth and message age, while batching, idempotent processing, retry policies, and dead-letter queues protect the system from transient failures.

9.5 Architectural Evolution Summary

The 20× scenario does not imply that every component should simply be multiplied by 20. Stateless compute workloads can scale horizontally, whereas stateful matching, write-intensive telemetry storage, relational transaction processing, and network egress require targeted architectural changes.

The proposed evolution preserves the eight service boundaries and exclusive data ownership established in Delivery 2. Capacity increases, partitioning, caching, asynchronous processing, and regional distribution will be applied according to each component's measured bottlenecks.

The 20× workload is therefore treated as a stress-test scenario that must be validated through load testing and capacity measurements before production deployment.

10 Cost Analysis

10.1 Estimation Assumptions

The monthly cost estimation considers two operating scenarios: the initial workload defined in Delivery 1 and the projected workload after a 20× increase.

The initial scenario is based on 45 million requests per hour (approximately 12,500 requests per second). The growth scenario considers 900 million requests per hour (250,000 requests per second).

For consistency, the estimate will use the following assumptions:

- All prices are expressed in USD and correspond to the selected AWS Region.
- A standard month of 730 operating hours is assumed for continuously running resources.
- On-Demand pricing is used as the baseline, without Reserved Instances, Savings Plans, or promotional credits.
- Multi-AZ deployments, replicas, backups, and data transfer charges are included where applicable.

- The growth scenario uses the resource configuration required to support the projected workload, rather than assuming that every cost category increases linearly by 20.

The estimates will be produced using the AWS Pricing Calculator, considering the resource configurations defined in the Compute, Networking, Data, Communication, Operations, and Security sections.[9]

10.2 Monthly Cost by Category

Table 11 summarizes the cost categories required for both workload scenarios. The final amounts must be obtained from the AWS Pricing Calculator after the corresponding resource configurations have been established.

Table 11: Estimated Monthly AWS Cost

Category	**Main Cost Drivers**	**Initial (USD/month)**	**20× Growth (USD/month)**
Compute	ECS tasks or EKS/EC2 instances, vCPU, memory, and running hours.	TBD	TBD
Databases	RDS instances, Multi-AZ deployment, storage, DynamoDB capacity, and backups.	TBD	TBD
Caching	ElastiCache node or serverless capacity and cluster configuration.	TBD	TBD
Storage	S3 storage volume, requests, lifecycle policies, and data retrieval.	TBD	TBD
Networking	NAT Gateways, data processing, data transfer, API Gateway, and load balancers.	TBD	TBD
Messaging	SQS requests, SNS publications and deliveries, and EventBridge events.	TBD	TBD
Monitoring	CloudWatch log ingestion and storage, metrics, alarms, Container Insights, and X-Ray traces.	TBD	TBD
Security	AWS WAF rules and requests, Secrets Manager secrets and API calls, and KMS usage.	TBD	TBD
Total		**TBD**	**TBD**

[9] https://calculator.aws/

10.3 Cost Calculation Methodology

Each category will be estimated according to its corresponding billing dimensions:

- **Compute:** For AWS Fargate, the estimate is based on the allocated vCPU-hours and GB-hours for all running tasks. For EC2-based deployments, instance type, quantity, and running hours are considered. EKS control-plane charges must also be included if EKS is selected.
- **Databases:** RDS costs include database instance-hours, storage, backup capacity, and Multi-AZ deployment. DynamoDB costs depend on read and write request units, item sizes, storage, and backup configuration.
- **Caching:** ElastiCache costs depend on the selected node type, number of nodes, cluster configuration, and operating hours.
- **Storage:** S3 costs depend on the amount of stored data, storage class, number of requests, and data retrieval.
- **Networking:** NAT Gateway costs include hourly usage and processed data. Load balancers, API Gateway requests, and data transfer between Availability Zones or Regions must also be considered.
- **Messaging:** SQS, SNS, and EventBridge costs depend on the number of requests, messages, event deliveries, and applicable message sizes.
- **Monitoring:** CloudWatch costs depend on ingested log volume, retained data, custom metrics, alarms, and Container Insights usage. X-Ray costs depend on recorded, retrieved, and analyzed traces.
- **Security:** AWS WAF costs depend on web ACLs, rules, and processed requests. Secrets Manager and KMS costs depend on stored secrets, API requests, keys, and cryptographic operations.

These billing dimensions are documented in the official AWS pricing pages for the corresponding services.[10]

[10] https://aws.amazon.com/fargate/pricing/; https://aws.amazon.com/rds/pricing/; https://aws.amazon.com/dynamodb/pricing/; https://aws.amazon.com/elasticache/pricing/; https://aws.amazon.com/vpc/pricing/; https://aws.amazon.com/sqs/pricing/; https://aws.amazon.com/sns/pricing/; https://aws.amazon.com/es/eventbridge/pricing/; https://aws.amazon.com/cloudwatch/pricing/; https://aws.amazon.com/secrets-manager/pricing/; https://aws.amazon.com/es/waf/pricing/

10.4 Cost Comparison and Interpretation

The growth scenario is expected to increase costs across compute, database operations, messaging, monitoring, and network traffic. However, the final increase will depend on the scaling characteristics of each resource.

Stateless compute services can scale according to demand, while managed databases may require larger instances, additional replicas, or different partitioning strategies. Similarly, telemetry ingestion and distributed tracing can significantly increase storage, processing, and monitoring expenses.

Consequently, the 20× scenario must be estimated using the actual resource quantities and usage assumptions established in the other sections, rather than applying a uniform multiplier to the initial monthly cost.

The final estimate will include the AWS Pricing Calculator export or screenshots, the selected Region, resource quantities, usage assumptions, pricing page URLs, and the date on which prices were consulted.